Unit 13: Strings

CONTENTS

Objectives
Introduction
13.1 Strings
13.2 Declaring and Initializing String
13.3 Reading and Writing Strings
13.4 Build-in-Library Functions to Manipulate Strings
13.5 strlen( )
13.6 strcpy( )
13.7 strcat( )
13.8 strcmp( )
13.9 Putting String Together
13.10 Comparison of two String
13.11 String Handling Functions
Summary
Keywords
Self Assessment
Answers for Self Assessment
Review Questions
Further Readings

Objectives

After studying this unit, you will be able to:

- Explain strings
- Describe reading and writing strings
- Explain string handling functions

Introduction

Computers process a variety of data kinds in addition to numeric data. The data to be processed is frequently textual, such as words, names, and addresses. String type variables are used to store and process this type of data. There is no explicit string data type in C.

Character arrays, on the other hand, can be used to simulate the same thing. In this session, we'll look at strings and how to manipulate them in C.

13.1 Strings

In C, a string is defined as a collection of characters. The NULL character, which signifies the end of the string, is used to end each string. Any group of characters enclosed in double-quote marks is referred to as a string constant.

When characters in a string constant are stored, the NULL character is automatically appended to the end of them. The escape sequence '0' is used to represent the NULL character within a programme. A string constant is an array with a lower bound of 0 and an upper bound of the string's length in characters.

When choosing a data representation for a given data object, the cost of performing various operations with that representation must be considered. Furthermore, a hidden cost resulting from the required storage management procedures must be considered.

Strings are stored in three types of structures:

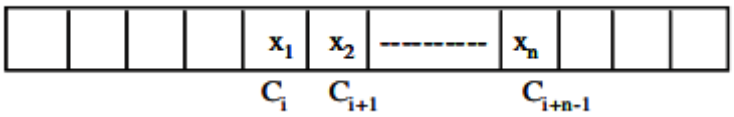
1. Fixed length structure

2. Variable length structure

3. Linked structure

4. Sequential Fixed Length Structure

In this representation successive characters of a string will be placed in consecutive character positions. The string S = 'x1---xn' could then be represented as in Figure with s as a pointer to the first character.

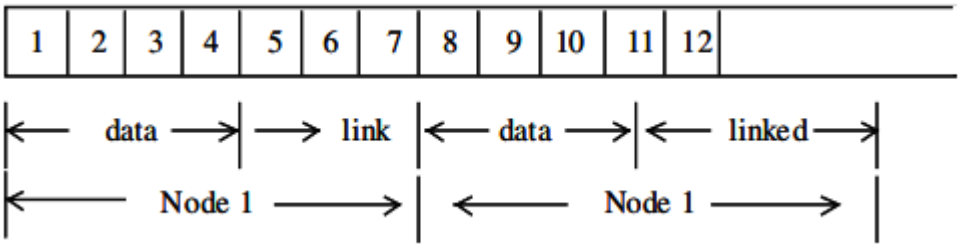


array

Now, if we want to pick a substring of size k from the string of size n, the time required to achieve this would be O(k) plus the time needed to locate a free space big enough to hold the string.

Linked List Fixed Size Nodes

The available memory is divided into nodes of fixed size. Each node has two fields: Data andLink. The size of a node is number of characters that can be stored in the DATA fields.



In the above figure memory is divided into nodes of size 4 with a link field that is two characters long. Deletion of a substring can be carried out by replacing all characters in this substring by 0 and freeing nodes in which the data fields consist of only 0's.

Storage compaction can be carried out when there are no free nodes. String representation with variable size is similar.

Each node in the purest version of a linked list representation of strings would be one in size. Normally, this would be considered a huge waste of space. With a two-character link field, this means that only 1/3 of the available Memory will be used to store string data, while the remaining 2/3 will be used exclusively for link data.

13.2 Declaring and Initializing String

In C, strings are represented as character arrays. The null character, which is just the character with the value 0, is used to indicate the end of the string. (The null character is unrelated to the null pointer except by name.) The null character is known as NUL in the ASCII character set.) Another character escape sequence, 0 represents the null or string-terminating character.

Because C has no built-in facilities for manipulating entire arrays (copying them, comparing them, etc.), it also has very few built-in facilities for manipulating strings.

In fact, C's only truly built-in string-handling is that it allows us to use string constants (also called string literals) in our code. Whenever we write a string, enclosed in double quotes, C automatically creates an array of characters for us, containing that string, terminated by the \0 character.

 **Example:** We can declare and define an array of characters, and initialize it with a string constant:

```
char string[] = "Hello, world!";
```

In this situation, we may omit the array's dimension because the compiler will figure it out for us based on the size of the initializer. The compiler will only size a string array for us in this scenario; in all other circumstances, we will have to decide how big the arrays and other data structures we employ to house strings are

To do anything else with strings, we must typically call functions. The C library contains a few basic string manipulation functions, and to learn more about strings, we'll be looking at how these functions might be implemented.

Since C never lets us assign entire arrays, we use the strcpy function to copy one string to another:

```
#include <string.h>
char string1[] = "Hello, world!";
char string2[20];
strcpy(string2, string1);
```

The destination string is strcpy's first argument, so that a call to strcpy mimics an assignment expression (with the destination on the left-hand side). Notice that we had to allocate string2 big enough to hold the string that would be copied to it. Also, at the top of any source file where we're using the standard library's string-handling functions (such as strcpy) we must include the line

```
#include <string.h>
```

which contains external declarations for these functions.

Since C won't let us compare entire arrays, either, we must call a function to do that, too. The standard library's strcmp function compares two strings, and returns 0 if they are identical, or a negative number if the first string is alphabetically "less than'' the second string, or a positive number if the first string is "greater." (Roughly speaking, what it means for one string to be "less than'' another is that it would come first in a dictionary or telephone book, although there are a few anomalies.) Here is an example:

```
char string3[] = "this is";
char string4[] = "a test";
if(strcmp(string3, string4) == 0)
printf("strings are equal\n");
elseprintf("strings are different\n");
```

This code fragment will print "strings are different''. Notice that strcmp does not return a Boolean,

true/false, zero/nonzero answer, so it's not a good idea to write something like

```
if(strcmp(string3, string4))
...
```

because it will behave backwards from what you might reasonably expect. (Nevertheless, if you start reading other people's code, you're likely to come across conditionals like if(strcmp(a, b)) or even if(!strcmp(a, b)). The first does something if the strings are unequal; the second does something if they're equal. You can read these more easily if you pretend for a moment that strcmp's name were strdiff, instead.)

Another standard library function is strcat, which concatenates strings. It does not concatenate two strings together and give you a third, new string; what it really does is append one string onto the end of another. (If it gave you a new string, it would have to allocate memory for it somewhere, and the standard library string functions generally never do that for you automatically.) Here's an example:

```
char string5[20] = "Hello, ";
char string6[] = "world!";
printf("%s\n", string5);
strcat(string5, string6);
printf("%s\n", string5);
```

The first call to printf prints ``Hello, '', and the second one prints "Hello, world!", indicating that the contents of string6 have been tacked on to the end of string5. Notice that we declared string5 with extra space, to make room for the appended characters.

If you have a string and you want to know its length (perhaps so that you can check whether it will fit in some other array you've allocated for it), you can call strlen, which returns the length of the string (i.e. the number of characters in it), not including the \0:

```
char string7[] = "abc";
intlen = strlen(string7);
Printf ("%d\n", len);
```

Finally, you can print strings out with printf using the %s format specifier, as we've been doingin these examples already (e.g. printf("%s\n", string5);).

Since a string is just an array of characters, all of the string-handling functions we've just seen can be written quite simply, using no techniques more complicated than the ones we already know. In fact, it's quite instructive to look at how these functions might be implemented. Here is a version of strcpy:

```
mystrcpy(char dest[], char src[])
{
inti = 0;
while(src[i] != '\0')
{
dest[i] = src[i];
i++;
}
dest[i] = '\0';
}
```

We've called it mystrcpy instead of strcpy so that it won't clash with the version that's already in the standard library. Its operation is simple: it looks at characters in the src string one at a time, and as long as they're not \0, assigns them, one by one, to the corresponding positions in the dest string. When it's done, it terminates the dest string by appending a \0. (After exiting the while loop, i is guaranteed to have a value one greater than the subscript of the last character in src.) For comparison, here's a way of writing the same code, using a for loop:

```
for(i = 0; src[i] != '\0'; i++)
dest[i] = src[i];
dest[i] = '\0';
```

Yet a third possibility is to move the test for the terminating \0 character out of the for loop header and into the body of the loop, using an explicit if and break statement, so that we can perform the test after the assignment and therefore use the assignment inside the loop to copy

the \0 to dest, too:

```
for(i = 0; ; i++)
{
dest[i] = src[i];
if(src[i] == '\0')
break;
}
```

(There are in fact many, many ways to write strcpy. Many programmers like to combine the assignment and test, using an expression like (dest[i] = src[i]) != '\0'. Here is a version of

strcmp:

```
mystrcmp(char str1[], char str2[])
{
inti = 0;
while(1)
{
if(str1[i] != str2[i])
return str1[i] - str2[i];
if(str1[i] == '\0' || str2[i] == '\0')
return 0;
i++;
}
}
```

Characters are compared one at a time. If two characters in one position differ, the strings are different, and we are supposed to return a value less than zero if the first string (str1) is alphabetically less than the second string. Since characters in C are represented by their numeric character set values, and since most reasonable character sets assign values to characters in alphabetical order, we can simply subtract the two differing characters from each other: the expression str1[i] - str2[i] will yield a negative result if the i'th character of str1 is less than the corresponding character in str2. (As it turns out, this will behave a bit strangely when comparing upper- and lower-case letters, but it's the traditional approach, which the standard versions of strcmp tend to use.) If the characters are the same, we continue around the loop, unless the characters we just compared were (both) \0, in which case we've reached the end of both strings, and they were both equal. Notice that we used what may at first appear to be an infinite loop—the controlling expression is the constant 1, which is always true. What actually happens is that the loop runs until one of the two return statements breaks out of it (and the entire function).

Finally, here is a version of strlen:

```
intmystrlen(char str[])
{
inti;
for(i = 0; str[i] != '\0'; i++)
{}
returni;
}
```

In this case, all we have to do is find the \0 that terminates the string, and it turns out that the three control expressions of the for loop do all the work; there's nothing left to do in the body.

Therefore, we use an empty pair of braces {} as the loop body. Equivalently, we could use a null statement, which is simply a semicolon:

```
for(i = 0; str[i] != '\0'; i++)
```

Empty loop bodies can be a bit startling at first, but they're not unheard of. Everything we've looked at so far has come out of C's standard libraries. As one last example, let's write a substr function, for extracting a substring out of a larger string. We might call it like this:

```
char string8[] = "this is a test";
char string9[10];
substr(string9, string8, 5, 4);
printf("%s\n", string9);
```

The idea is that we'll extract a substring of length 4, starting at character 5 (0-based) of string8, and copy the substring to string9. Just as with strcpy, it's our responsibility to declare the destination string (string9) big enough. Here is an implementation of substr. Not surprisingly, it's quite similar to strcpy:

```
substr(char dest[], char src[], int offset, intlen)
{
inti;
for(i = 0; i<len&&src[offset + i] != '\0'; i++)
dest[i] = src[i + offset];
dest[i] = '\0';
}
```

If you compare this code to the code for mystrcpy, you'll see that the only differences are that characters are fetched from src[offset + i] instead of src[i], and that the loop stops when len characters have been copied (or when the src string runs out of characters, whichever comesfirst).

In this unit, we've been careless about declaring the return types of the string functions, and(with the exception of mystrlen) they haven't returned values. The real string functions do return values, but they're of type ``pointer to character,'' which we haven't discussed yet. When working with strings, it's important to keep firmly in mind the differences between characters and strings. We must also occasionally remember the way characters are represented, and about the relation between character values and integers.

As we have had several occasions to mention, a character is represented internally as a small integer, with a value depending on the character set in use. For example, we might find that 'A' had the value 65, that 'a' had the value 97, and that '+' had the value 43. (These are, in fact, the values in the ASCII character set, which most computers use. However, you don't need to learn these values, because the vast majority of the time, you use character constants to refer to characters, and the compiler worries about the values for you. Using character constants in preference to raw numeric values also makes your programs more portable.)

As we may also have mentioned, there is a big difference between a character and a string, even a string which contains only one character (other than the \0).

Example: 'A' is not the same as "A". To drive home this point, let's illustrate it with a few examples.

If you have a string:

```
char string[] = "hello, world!";
```

you can modify its first character by saying

```
string[0] = 'H';
```

(Of course, there's nothing magic about the first character; you can modify any character in the string in this way. Be aware, though, that it is not always safe to modify strings in-place like this;

we'll say more about the modifiability of strings in a later unit on pointers.) Since you're replacing a character, you want a character constant, 'H'. It would not be right to write

string[0] = "H"; /* WRONG */

because "H" is a string (an array of characters), not a single character. (The destination of the assignment, string[0], is a char, but the right-hand side is a string; these types don't match.) On the other hand, when you need a string, you must use a string. To print a single newline, you could call

printf("\n");

It would not be correct to call

printf('\n'); /* WRONG */

printf always wants a string as its first argument. (As one final example, putchar wants a single character, so putchar('\n') would be correct, and putchar("\n") would be incorrect.) We must also remember the difference between strings and integers. If we treat the character '1' as an integer, perhaps by saying

inti = '1';

we will probably not get the value 1 in i; we'll get the value of the character '1' in the machine's character set. (In ASCII, it's 49.) When we do need to find the numeric value of a digit character (or to go the other way, to get the digit character with a particular value) we can make use of the fact that, in any character set used by C, the values for the digit characters, whatever they are, are contiguous. In other words, no matter what values '0' and '1' have, '1' - '0' will be 1 (and, obviously, '0' - '0' will be 0). So, for a variable c holding some digit character, the expression

c - '0'

gives us its value. (Similarly, for an integer value i, i + '0' gives us the corresponding digit character, as long as 0 <= i<= 9.)

Just as the character '1' is not the integer 1, the string "123" is not the integer 123. When we have a string of digits, we can convert it to the corresponding integer by calling the standard function

atoi:

char string[] = "123";

inti = atoi(string);

int j = atoi("456");

13.3 Reading and Writing Strings

Character String Input Function

% ws or % wc can be used as the specification for reading character strings. The specifier % terminates reading a string at the encounter of blank space. Some versions of scanf( ) support the following conversion specification for strings.

% [characters] and % [^characters]

The specification % [characters] means that only the characters specified within the brackets are permissible in the input string. If the input string contains any other character, the string will be terminated at the first encounter of such a character.

The specification % [^character] does exactly the reverse, i.e., characters specified after circumflex (^) are not permitted.

gets( ) function is used to read a character entered at the keyboard and places it at the address pointed to by its character pointer argument.

Characters are entered until the enter key is pressed.

syntax: char * gets (char *a);

where a is the character array.

Character String Output Function

puts( ) function writes its string argument to the screen followed by the new line.

syntax: char * puts (const char * a);

puts( ) function takes less space thenprintf( ). It is faster than printf( ). It does not output numbers or does format conversions as puts( ) outputs character string only.

Example:

```
# include <stdio.h>
# include <conio.h>
main( )
{
charstr [50]
gets (str);
puts (str);
}
```

13.4 Build-in-Library Functions to Manipulate Strings

With every C compiler a large set of useful string handling library functions are provided.

Table lists the more commonly used functions along with their purpose.

Functions	Use
strlen	Finds length of a string
strlwr	Converts a string to lowercase
strupr	Converts a string to uppercase
strcat	Appends one strings to uppercase
strncat	Appends first n characters of a string at the end of another
strcpy	Copies a string into another
strncpy	Copies first n characters o one string into another
strcmp	Compares two strings
strncmp	Compares first n characters of two strings
strcmpi	Compares two strings without regard to case ("I" denotes that this function ignores case)
stricmp	Compares two strings without regards to case (identical to strcmpi)
strnicmp	Compares first n characters if two strings without regard to case
strdup	Duplicates a string
strchr	Finds first occurrence of a given character in a string
strchr	Finds last occurrence of a given character in a string
strrchr	Finds last occurrence of a given character in a string
strstr	Finds first occurrence of a given striung in another string
strset	Sets all characers of string to a given character
strnset	Sets first n characters of a string to a given character
strrev	Reverses string

Out of the above list, We shall discuss the functions strlen( ), strcpy( ), strcat( ) and strcmp( ), since these are the most commonly used functions. This will also illustrate how the library functions in general handle strings. Let us study these functions one by one.

13.5 strlen( )

This function counts the number of characters present in a string. Its usage is illustrated in the following program.

```
main( )
{
chararr[ ] = "Bamboozled" ;
int len1, len2 ;
len1 = strlen( arr ) ;
len2 = strlen( "Humpty Dumpty" ) ;
printf ( "\nstring = %s length = %d", arr, len1 ) ;
printf ( "\nstring = %s length = %d", "Humpty Dumpty", len2 ) ;
}
```

The output would be...

string = Bamboozled length = 10

string = Humpty Dumpty length = 13

13.6 strcpy( )

This function copies the contents of one string into another. The base addresses of the source and target strings should be supplied to this function. Here is an example of strcpy( ) in action...

Lab Exercise:

```
main( )
{
char source[ ] = "Sayonara" ;
char target[20] ;
strcpy ( target, source ) ;
printf ( "\nsource string = %s", source ) ;
printf ( "\ntarget string = %s", target ) ;
}
```

And here is the output...

source string = Sayonara

target string = Sayonara

On supplying the base addresses, strcpy( ) goes on copying the characters in source string into the target string till it doesn't encounter the end of source string ('\0'). It is our responsibility to see to it that the target string's dimension is big enough to hold the string being copied into it.

Thus, a string gets copied into another, piece-meal, character by character. There is no short cut for this. Let us now attempt to mimic strcpy( ), via our own string copy function, which we will call xstrcpy( ).

Lab Exercise:

```
main( )
{
char source[ ] = "Sayonara" ;
```

```
        char target[20] ;
        xstrcpy ( target, source ) ;
        printf ( "\nsource string = %s", source ) ;
        printf ( "\ntarget string = %s", target ) ;
        }
        xstrcpy ( char *t, char *s )
        {
        while ( *s != '\0' )
        {
        *t = *s ;
        s++ ;
        t++ ;
        }
        *t = '\0' ;
}
```

The output of the program would be...

source string = Sayonara

target string = Sayonara

13.7 strcat( )

This function concatenates the source string at the end of the target string. For example, "Bombay"and "Nagpur" on concatenation would result into a string "Bombay Nagpur". Here is an example of strcat( ) at work.

Lab Exercise:

```
        main( )
        {
        char source[ ] = "Folks!" ;
        char target[30] = "Hello" ;
        strcat ( target, source ) ;
        printf ( "\nsource string = %s", source ) ;
        printf ( "\ntarget string = %s", target ) ;
        }
```

And here is the output...

source string = Folks!

target string = HelloFolks!

13.8 strcmp( )

This is a function which compares two strings to find out whether they are same or different. The two strings are compared character by character until there is a mismatch or end of one of the strings is reached, whichever occurs first. If the two strings are identical, strcmp( ) returns a value zero. If they're not, it returns the numeric difference between the ASCII values of the first non-matching pairs of characters. Here is a program which puts strcmp( ) in action.

Lab Exercise:

```
main( )
{
char string1[ ] = "Jerry" ;
char string2[ ] = "Ferry" ;
inti, j, k ;
i = strcmp( string1, "Jerry" ) ;
j = strcmp( string1, string2 ) ;
k = strcmp( string1, "Jerry boy" ) ;
printf ( "\n%d %d %d", i, j, k ) ;
}
```

And here is the output...

0 4 -32

13.9 Putting String Together

```
#include <string.h>
#include <stdio.h>
int main()
{
char first[100];
char last[100];
charfull_name[200];
strcpy(first, "firstName");
strcpy(last, "secondName");
strcpy(full_name, first);
strcat(full_name, " ");
strcat(full_name, last);
printf("The full name is %s\n", full_name);
return (0);
}
```

13.10 Comparison of two String

```
#include <string.h>
i = strcmp( s1, s2 );
```

Where:

```
const char *s1, *s2;
```

are the strings to be compared.

```
inti;
```

gives the results of the comparison. "i" is zero if the strings are identical. "i" is positive if string "s1" is greater than string "s2", and is negative if string "s2" is greater than string "s1". Comparisons of "greater than" and "less than" are made according to the ASCII collating sequence.

13.11 String Handling Functions

A close analysis of the essential string-handling facilities required of any text creation and editing system (formal or otherwise) should lead to the following list of primitive functions:

1. Create a string of test
2. Concatenate two strings to form another string
3. Search and replace (if desired) a given substring within a string
4. Test for the identity of a string
5. Compute the length of a string

String related functions are grouped into string.h header file. It contains the following functions among others:

1. char * strcat(char *dest, const char *src): This function appends one string to another returning a pointer to concatenated string. It appends a copy of src to the end of dest. The length of the resulting string is strlen(dest) + strlen(src). strings.

2. intstrcmp(const char *s1, const char *s2): This function compares two strings. The string comparison starts with the first character in each string and continues with subsequent characters until the corresponding characters differ or until the end of the strings is reached.

The returned values are integers as follows:

< 0 if s1 < s2

= 0 if s1 = = s2

> 0 if s1 > s2

3. char *strcpy(char *dest, const char *src): This function copies string src to dest stopping after the terminating null character has been moved. The return value is dest.

4. intstrlen(const char *s): This function returns the length of a string (i.e., the number of characters in s), not counting the terminating null character.

5. intstrncmp(const char *s1, const char *s2, intmaxlen): This function compares portions of two strings s1 and s2 looking at no more than maxlen characters. The string comparison starts with the first character in each string and continues with subsequent characters until the corresponding characters differ or until maxlen characters have been examined. It returns an int value based on the result of comparing s1 (or part of it) to s2 (or part of it) as

given below:

< 0 if s1 < s2

= 0 if s1 == s2

> 0 if s1 > s2

Summary

- A string is defined in C as an array of characters. Each string is terminated by the NULL character, which indicates end of the string.
- A string constant is denoted by any set of characters included in double-quote marks.
- The NULL character is automatically appended to the end of the characters in a string constant when they are stored. Within a program, the NULL character is denoted by the escape sequence '\ 0'.
- A string constant represents an array whose lower bound is 0 and whose upper bound is the number of characters in the string.

- Strings are stored in three types of structures - Fixed length structure, Variable length structure, and Linked structure.

Keywords

gets( ): A C library function used to read a character entered at the keyboard and to place it at theaddress pointed to by its character pointer argument

puts( ): A C library function that writes its string argument to the screen followed by the newline

strcat(): The C library function that appends one string to another returning a pointer to concatenated string

strcmp(): The C library function that compares two strings

string.h: A C header file that contains string manipulating library functions

String: An array of characters terminated by the NULL character

Self Assessment

1. Which one is correct method for Initializing string?

A. char abc[ ] =“hello”;
B. char abc[10]= {‘h’,’e’,’l’,’l’,’o’,’\0’};
C. Char abc[ ] = {‘h’,’e’,’l’,’l’,’o’,’\0’};
D. above all

2. Which method is use to read string ?

A. Puts ( )
B. Gets ( )
C. Print ( )
D. None of above

3. Which method is use to write string?

A. Scanf ( )
B. Gets ( )
C. Puts ( )
D. Printf ( )

4. String is an_____

A. an array of characters
B. sequence of characters terminated with a null character
C. both a and b
D. none of above

5. Which of the following function is more appropriate for reading in a multi-word string?

A. gets ( )
B. Puts ( )
C. Printf ( )
D. Sizeo ( )

6. Format specifier is used to print a string is _____

A. % d
B. % c
C. % s
D. % f

7. Is there any difference between the two statements?

char *ch = "string ";

charch[] = "String";

A. Yes
B. No

8. Function of strcat ( ) is_____

A. compares two strings
B. converts string to lowercase
C. concatenates (joins) two strings
D. none of above

9. If the two strings are identical, then strcmp( ) function returns

A. 1
B. 0
C. 2
D. -1

10. The library function used to computes string's length

A. strlwr ( )
B. strcpy ( )
C. strlen ( )
D. strpr ( )

11. What will strupr( ) function do?

A. compares two strings
B. converts string to uppercase
C. computes string's length
D. none of above

12. Function of strcpy ( ) is_____

A. converts string to uppercase
B. sequence of characters terminated with a null character
C. copies a string to another
D. none of above

13. What is the maximum length of a C String?

A. 16 characters
B. 8 characters
C. 32 characters
D. None of above

14. What will strlwr( ) function do?

A. converts string to lowercase
B. converts string to uppercase
C. computes string's length
D. none of above

15. Find incorrect statement _____

A. char input[100];
B. puts('Input string');
C. gets(input);
D. puts("Entered string is");

Answers for Self Assessment

1.	D	2.	B	3.	C	4.	C	5.	C
6.	C	7.	A	8.	C	9.	B	10.	C
11.	B	12.	C	13.	D	14.	A	15.	B

Review Questions

1. Write a C program that reads a sentence from the keyboard and prints the frequency of each letter.
2. How can you create a string type C variable? Can they be assigned to each other in the same way as other data types? Explain.
3. Write a program that converts a string like "124" to an integer 124.
4. Write a program that replaces two or more consecutive blanks in a string by a single blank.

   For example, if the input is

   Grim return to the planet of apes!!

   the output should be

   Grim return to the planet of apes!!
5. Can an array of pointers to strings be used to collect strings from the keyboard? If not, why not?
6. Write a program to sort a set of names stored in an array in alphabetical order.
7. Write a program to delete all vowels from a sentence. Assume that the sentence is not more than 80 characters long.
8. Write a program that takes a set of names of individuals and abbreviates the first, middle and other names except the last name by their first letter.

Further Readings

Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi

Byron Gottfried, "Programming With C", Tata McGraw Hill Publishing Company Limited, New Delhi

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

YashvantKanetkar, Let us C

Web Links

www.en.wikipedia.org

www.web-source.net

www.webopedia.com